Algoritmo

Leandro Olguin

2026-04-01

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see http://rmarkdown.rstudio.com.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

Primeros comandos de R

R es un lenguaje muy parecido a mathlab y permite el trabajo con matrices.

```
a=38
a=40
b=80
c=b-a
c
```

```
## [1] 40
```

```
y=40
alpha<-30
```

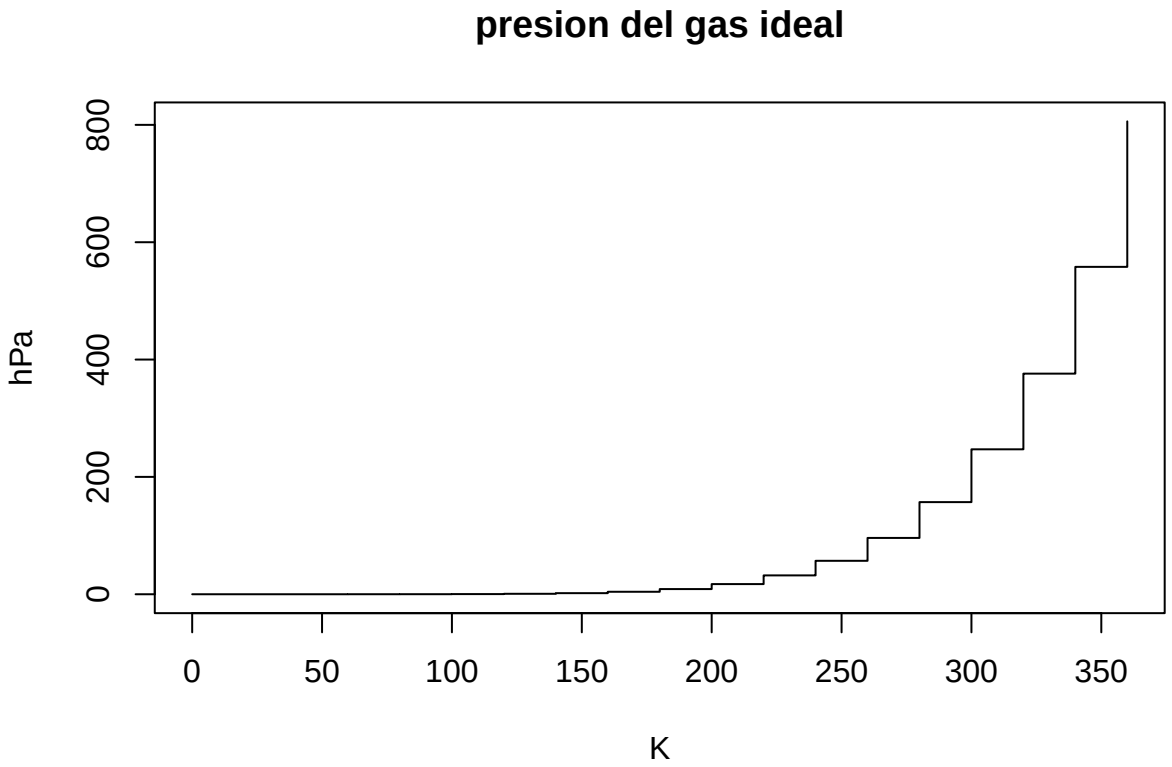
##uso de datasets o base de datos internos de R usando el comando data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan dados en tablas de las cuales podemos elegir columnas especificas si escribimos $ depues del comando de informacion especifico y elegimos la columa de datos deseada

```
summary(cars)
```

```
##      speed           dist       
##  Min.   : 4.0   Min.   :  2.00  
##  1st Qu.:12.0   1st Qu.: 26.00  
##  Median :15.0   Median : 36.00  
##  Mean   :15.4   Mean   : 42.98  
##  3rd Qu.:19.0   3rd Qu.: 56.00  
##  Max.   :25.0   Max.   :120.00
```

Including Plots

You can also embed plots, for example:



##comando plot El comando plot es un comando que permite graficar cualquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar un comando podemos escribir ##plot## en la consola, lo cual abrira una ventana que explicara en detalle el comando.

##Funciones estadisticas

```
z1<-rnorm(350,22,5)
z1
```

```
##   [1] 21.761915 31.235985 23.061729 23.847939 29.473239 14.212591 19.455991
##   [8] 24.889917 17.643254 24.356736 24.052454 23.821865 18.086339 22.037372
##  [15] 13.630671 28.824778 20.481452 18.116245 15.338755 26.632652 23.618757
##  [22] 20.981086 20.976727 21.013149 17.109319 15.406083 22.822322 24.451497
##  [29] 28.722901 20.446809 11.333198 21.661689 24.921031 19.725552 19.617108
##  [36] 25.756043 17.510668 21.579860 18.305001 22.133944 24.394006 12.744917
##  [43] 22.417675 16.114224 25.307958 29.134823 24.024255 17.919422 23.680382
##  [50] 21.491478 24.197983 26.488755 29.994096 29.584901 21.085336 23.614968
##  [57] 23.169320 26.567150 26.823113 19.980671 22.244427 21.761635 30.975792
##  [64] 21.868203 22.671812 20.903746 21.765050 32.192369 21.638849 16.242511
##  [71] 12.972777 25.335171 24.336270 12.413782 19.241886 23.365754 22.106393
##  [78] 19.000006 19.829298  8.580565 16.706056 22.998405 23.981397 18.092742
##  [85] 27.372403 22.857307 24.231399 19.424797 17.283043 24.337166 26.806435
##  [92] 20.251347 22.102810 27.247884 21.110687 30.359086 29.974146 19.913823
##  [99] 25.470757 18.837402 25.437179 24.074616 24.420434 19.758976 20.816777
## [106] 18.335805 32.026008 22.936000 28.362526 21.604243 29.122285 11.718717
## [113] 12.601344 15.671912 23.065278 20.064084 18.028700 22.821608 24.984725
## [120] 14.413742 15.198695 21.743376 21.052924 21.003673 33.023264 21.966318
## [127] 13.551369 13.241861 28.833432 27.288005 18.621078 20.025602 28.855632
## [134] 17.482030 17.274740 27.065190 30.955782 22.478703 17.956645 22.258143
## [141] 21.695588 23.834154 15.157238 23.973025 25.007809 12.964780 19.250293
```

```
## [148] 22.701685 20.470935 26.864102 16.383300 17.919671 24.900371 15.451706
## [155] 31.627656 22.736619 23.655481 37.323924 26.356411 24.091854 21.811360
## [162] 18.309633 27.102632 16.692056 13.056403 23.714550 19.943374 16.155298
## [169] 23.029449 29.943025 24.735793 21.343554 18.203066 22.412246 21.657137
## [176] 18.503797 20.047654 16.371004 15.724811 17.858294 28.373449 21.595739
## [183] 29.046024 22.756525 16.394321  5.761313 24.877788 24.222279 25.939948
## [190] 22.495932  9.387748 15.764445 17.716988 14.833310 20.948891 19.679059
## [197] 26.184183 23.901263 17.029352 22.358052 23.235878 26.402203 27.045355
## [204] 26.174641 30.429355 16.928407 20.809787 16.271211 26.792744 17.152900
## [211] 21.746663 19.425713 27.058551 20.753549 26.167035 18.105367 18.619109
## [218] 14.536530 22.992474 26.155442 17.085150 21.807590 29.402183 16.998813
## [225] 27.345565  8.068161 22.835337 23.668897 11.843448 14.350467 28.580734
## [232] 21.845951 18.373263 27.069095 18.229590 13.488430 19.418371 30.252560
## [239] 17.416662 21.574218 18.690997 13.700596 16.363843 24.261276 20.773892
## [246] 23.061808 25.928038 20.337310 19.448138 17.888253 18.634794 19.229273
## [253] 18.629720 21.624828 27.954571 34.238458 19.029293 13.566065 17.341912
## [260] 25.899700 27.004772 23.850772 21.332990 28.290232 20.010999 25.685298
## [267] 17.379082 15.182428 22.914612 28.154904 13.213093 19.852554 20.076031
## [274] 20.002380 22.353296 27.930315 27.755045 26.475879 19.287734 18.126977
## [281] 17.817139 14.611895 23.683265 23.553856 24.265737 27.197906 26.677544
## [288] 19.823012 15.343846 20.195867 20.873653 33.132238 24.367358 23.600129
## [295] 27.216838 24.163376 17.127937 24.922518 11.533068 22.627011 13.855349
## [302] 17.419557 26.703988 22.985133 17.947248 12.658023 25.651334 18.017753
## [309] 16.327305 34.080380 22.373698 26.425168 19.552729 20.464294 23.975000
## [316] 24.261715 12.781693 22.396841 21.540859 22.652857 19.696524 23.221145
## [323] 28.196626 32.428296 13.087650 21.539609 33.753429 32.208927 16.234641
## [330] 22.671656 21.239117 35.291988 17.705303 23.207157 24.735048 22.189736
## [337] 20.943678 22.988854 17.154872 21.020336 22.133150 22.332718 20.541954
## [344] 26.739675 21.237186 15.603790 13.658291 16.042504 26.473365 24.811802
```

```
w1<-length(z1)
w1
```

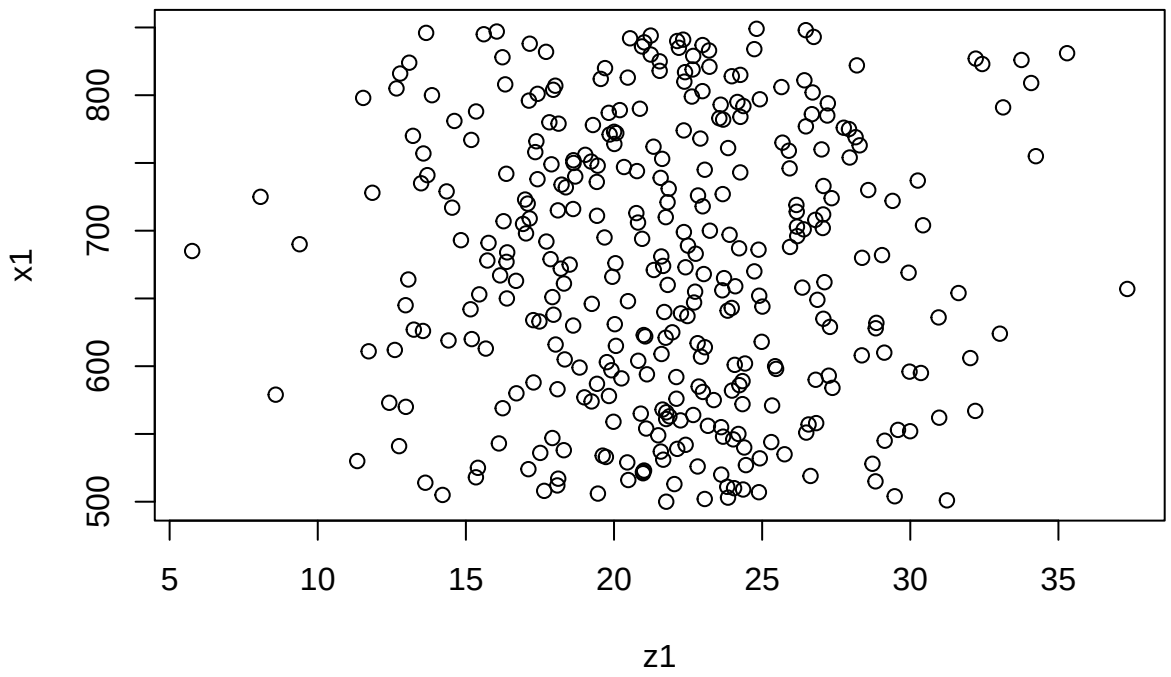
```
## [1] 350
```

```
x1<-(500:849)
x1
```

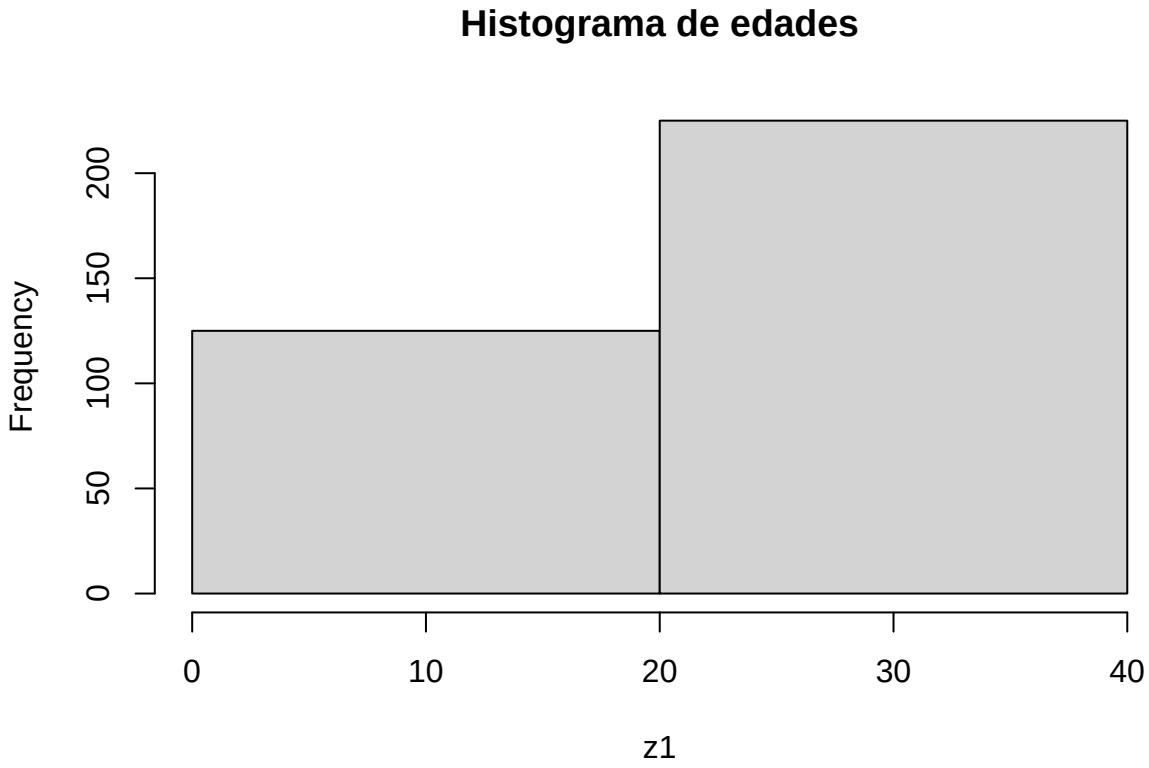
```
##   [1] 500 501 502 503 504 505 506 507 508 509 510 511 512 513 514 515 516 517
##  [19] 518 519 520 521 522 523 524 525 526 527 528 529 530 531 532 533 534 535
##  [37] 536 537 538 539 540 541 542 543 544 545 546 547 548 549 550 551 552 553
##  [55] 554 555 556 557 558 559 560 561 562 563 564 565 566 567 568 569 570 571
##  [73] 572 573 574 575 576 577 578 579 580 581 582 583 584 585 586 587 588 589
##  [91] 590 591 592 593 594 595 596 597 598 599 600 601 602 603 604 605 606 607
## [109] 608 609 610 611 612 613 614 615 616 617 618 619 620 621 622 623 624 625
## [127] 626 627 628 629 630 631 632 633 634 635 636 637 638 639 640 641 642 643
## [145] 644 645 646 647 648 649 650 651 652 653 654 655 656 657 658 659 660 661
## [163] 662 663 664 665 666 667 668 669 670 671 672 673 674 675 676 677 678 679
## [181] 680 681 682 683 684 685 686 687 688 689 690 691 692 693 694 695 696 697
## [199] 698 699 700 701 702 703 704 705 706 707 708 709 710 711 712 713 714 715
## [217] 716 717 718 719 720 721 722 723 724 725 726 727 728 729 730 731 732 733
## [235] 734 735 736 737 738 739 740 741 742 743 744 745 746 747 748 749 750 751
## [253] 752 753 754 755 756 757 758 759 760 761 762 763 764 765 766 767 768 769
## [271] 770 771 772 773 774 775 776 777 778 779 780 781 782 783 784 785 786 787
## [289] 788 789 790 791 792 793 794 795 796 797 798 799 800 801 802 803 804 805
## [307] 806 807 808 809 810 811 812 813 814 815 816 817 818 819 820 821 822 823
```

```
## [325] 824 825 826 827 828 829 830 831 832 833 834 835 836 837 838 839 840 841
## [343] 842 843 844 845 846 847 848 849
```

```
plot(z1,x1)
```




```
hist(z1,main="Histograma de edades",breaks=2)
```




```
density(z1,type='d')
```

```
## Warning: In density.default(z1, type = "d") :
```

```
##  extra argument 'type' will be disregarded
```

```
##
## Call:
##  density.default(x = z1, type = "d")
##
## Data: z1 (350 obs.); Bandwidth 'bw' = 1.403
##
##        x                y
##  Min.   : 1.552   Min.   :9.049e-06
##  1st Qu.:11.547   1st Qu.:1.658e-03
##  Median :21.543   Median :1.249e-02
##  Mean   :21.543   Mean   :2.496e-02
##  3rd Qu.:31.538   3rd Qu.:4.646e-02
##  Max.   :41.534   Max.   :8.256e-02
```

```
##plot(density(z1),type='d') ver si este plot va pq me ponia error
```

##Ejercicio 1 crear un vector que tenga todos los numeros enteros desde el 1 hasta 272

```
Secuencia_dni<-(1:272)
Secuencia_dni
```

```
##   [1]   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18
##  [19]  19  20  21  22  23  24  25  26  27  28  29  30  31  32  33  34  35  36
##  [37]  37  38  39  40  41  42  43  44  45  46  47  48  49  50  51  52  53  54
##  [55]  55  56  57  58  59  60  61  62  63  64  65  66  67  68  69  70  71  72
##  [73]  73  74  75  76  77  78  79  80  81  82  83  84  85  86  87  88  89  90
##  [91]  91  92  93  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108
## [109] 109 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125 126
## [127] 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141 142 143 144
## [145] 145 146 147 148 149 150 151 152 153 154 155 156 157 158 159 160 161 162
## [163] 163 164 165 166 167 168 169 170 171 172 173 174 175 176 177 178 179 180
## [181] 181 182 183 184 185 186 187 188 189 190 191 192 193 194 195 196 197 198
## [199] 199 200 201 202 203 204 205 206 207 208 209 210 211 212 213 214 215 216
## [217] 217 218 219 220 221 222 223 224 225 226 227 228 229 230 231 232 233 234
## [235] 235 236 237 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252
## [253] 253 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269 270
## [271] 271 272
```

##Ejercicio 2 calcular la suma de todos los valores del vector secuencia dni

```
Total<-1
Valor_final<-length(Secuencia_dni)
for (i in 1:Valor_final) {
  Total<-Total+i }
  Total
```

```
## [1] 37129
```

##Ejercicio 3 Repetir el ejercicio anterior en phyton

```
suma=0
for i in range (1,273):
  suma+=i
  print(suma)
```

```
## 1
## 3
```

```
## 6
## 10
## 15
## 21
## 28
## 36
## 45
## 55
## 66
## 78
## 91
## 105
## 120
## 136
## 153
## 171
## 190
## 210
## 231
## 253
## 276
## 300
## 325
## 351
## 378
## 406
## 435
## 465
## 496
## 528
## 561
## 595
## 630
## 666
## 703
## 741
## 780
## 820
## 861
## 903
## 946
## 990
## 1035
## 1081
## 1128
## 1176
## 1225
## 1275
## 1326
## 1378
## 1431
## 1485
## 1540
## 1596
```

```
## 1653
## 1711
## 1770
## 1830
## 1891
## 1953
## 2016
## 2080
## 2145
## 2211
## 2278
## 2346
## 2415
## 2485
## 2556
## 2628
## 2701
## 2775
## 2850
## 2926
## 3003
## 3081
## 3160
## 3240
## 3321
## 3403
## 3486
## 3570
## 3655
## 3741
## 3828
## 3916
## 4005
## 4095
## 4186
## 4278
## 4371
## 4465
## 4560
## 4656
## 4753
## 4851
## 4950
## 5050
## 5151
## 5253
## 5356
## 5460
## 5565
## 5671
## 5778
## 5886
## 5995
## 6105
```

```
## 6216
## 6328
## 6441
## 6555
## 6670
## 6786
## 6903
## 7021
## 7140
## 7260
## 7381
## 7503
## 7626
## 7750
## 7875
## 8001
## 8128
## 8256
## 8385
## 8515
## 8646
## 8778
## 8911
## 9045
## 9180
## 9316
## 9453
## 9591
## 9730
## 9870
## 10011
## 10153
## 10296
## 10440
## 10585
## 10731
## 10878
## 11026
## 11175
## 11325
## 11476
## 11628
## 11781
## 11935
## 12090
## 12246
## 12403
## 12561
## 12720
## 12880
## 13041
## 13203
## 13366
## 13530
```

```
## 13695
## 13861
## 14028
## 14196
## 14365
## 14535
## 14706
## 14878
## 15051
## 15225
## 15400
## 15576
## 15753
## 15931
## 16110
## 16290
## 16471
## 16653
## 16836
## 17020
## 17205
## 17391
## 17578
## 17766
## 17955
## 18145
## 18336
## 18528
## 18721
## 18915
## 19110
## 19306
## 19503
## 19701
## 19900
## 20100
## 20301
## 20503
## 20706
## 20910
## 21115
## 21321
## 21528
## 21736
## 21945
## 22155
## 22366
## 22578
## 22791
## 23005
## 23220
## 23436
## 23653
## 23871
```

```
## 24090
## 24310
## 24531
## 24753
## 24976
## 25200
## 25425
## 25651
## 25878
## 26106
## 26335
## 26565
## 26796
## 27028
## 27261
## 27495
## 27730
## 27966
## 28203
## 28441
## 28680
## 28920
## 29161
## 29403
## 29646
## 29890
## 30135
## 30381
## 30628
## 30876
## 31125
## 31375
## 31626
## 31878
## 32131
## 32385
## 32640
## 32896
## 33153
## 33411
## 33670
## 33930
## 34191
## 34453
## 34716
## 34980
## 35245
## 35511
## 35778
## 36046
## 36315
## 36585
## 36856
## 37128
```

##Ejercicio 4 Cuanto tarda en correr el codigo del ejercicio 2?

```
inicio<-Sys.time()
Total<-0
Valor_final<-length(Secuencia_dni)
for (i in 1:Valor_final) {
  Total<-Total+i }
  Total
```

```
## [1] 37128
```

```
  Sys.time()
```

```
## [1] "2026-05-19 03:23:08 UTC"
```

```
  final<-Sys.time()
  final-inicio
```

```
## Time difference of 0.007975101 secs
```

##Ejercicio 5 Repetir el ejercicio anterior usando la cabeza como Gauss

```
system.time({
  n <- length(Secuencia_dni)

  #Aplicamos la fórmula de Gauss
  total_gauss <- (n*(n+1))/2

  print(total_gauss)
})
```

```
## [1] 37128
```

```
##    user  system elapsed
##   0.001   0.000   0.000
```

##PARTE 2 ##1.4 CONSIGNA: Comparar la performance con systime

```
A <- 0
InicioA=Sys.time()
for (i in 1:50000) { A[i] <- (i*2)}
head (A)
```

```
## [1]  2  4  6  8 10 12
```

```
tail(A)
```

```
## [1]  99990  99992  99994  99996  99998 100000
```

```
FinA=Sys.time()
FinA-InicioA
```

```
## Time difference of 0.04034448 secs
```

```
InicioB=Sys.time()
B=seq(1, 100000, by = 2)
head(B)
```

```
## [1]  1  3  5  7  9 11
```

```
tail(B)
```

```
## [1] 99989 99991 99993 99995 99997 99999
```

```
FinB=Sys.time()
FinB-InicioB
```

```
## Time difference of 0.005519867 secs
```

##1.5 CONSIGNA: Implementacion de una serie Fibonachi o Fibonacci

```
n <- 30
system.time({
  fib <- numeric(n)
  fib[1] <- 0
  fib[2] <- 1
  for(i in 3:n){
    fib[i] <- fib[i-1]+fib[i-2]
  }
})
```

```
##    user  system elapsed
##   0.004   0.000   0.004
```

```
print("primeros numeros de la serie ")
```

```
## [1] "primeros numeros de la serie "
```

```
head(fib,10)
```

```
##  [1]  0  1  1  2  3  5  8 13 21 34
```

```
fib_recursiva<- function(k){
  if(k==0)return(0)
  if(k==1)return(1)
  return(fib_recursiva(k-1)+fib_recursiva(k-2))
}
print("tiempo de ejecucion recursiva:")
```

```
## [1] "tiempo de ejecucion recursiva:"
```

```
system.time({
  resultado_rec <- fib_recursiva(n)
})
```

```
##    user  system elapsed
##   1.109   0.006   1.564
```

##1.6 CONSIGNA: Cuantas iteraciones se necesitan para generar un número de la serie mayor que 1.000.000

```
a0=0
a1=1
it=2
  while (a1<1000001){
    an=a0+a1
    a0=a1
    a1=an
    it=it+1}
cat("el primer numero mayor a 1000000 es:",a1,"/n")
```

```
## el primer numero mayor a 1000000 es: 1346269 /n
```

```
cat("se necesitaron", it-1,"iteraciones (pasos) para superarlo.")
```

```
## se necesitaron 31 iteraciones (pasos) para superarlo.
```

```
it
```

```
## [1] 32
```

##1.7 CONSIGNA: Compara la performance de ordenación del método burbuja vs el método sort de R Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```
library(microbenchmark)
x<-sample(1:20000,10)
# Creo una funcion para ordenar
burbuja <- function(x){
  res <- microbenchmark(NULL, burbuja, times=1000L)
n<-length(x)
for(j in 1:(n-1)){
for(i in 1:(n-j)){
if(x[i]>x[i+1]){
temp<-x[i]
x[i]<-x[i+1]
x[i+1]<-temp
}
}
}
return(x)
}
res<-burbuja(x)
#Muestra obtenida
x
```

```
##  [1]  7115  3150 17635  4501  4686 17423  7857 15571  2294  7908
```

```
res
```

```
##  [1]  2294  3150  4501  4686  7115  7857  7908 15571 17423 17635
```

```
sort(x)
```

```
##  [1]  2294  3150  4501  4686  7115  7857  7908 15571 17423 17635
```

1.8 La penitencia de Newton

```
n <- 1e7
system.time({
suma_iterativa <- 0
for (i in 1:n) {
suma_iterativa <- suma_iterativa + i
}
})
```

```
##    user  system elapsed
##   0.226   0.000   0.245
```

```
cat("Resultado Suma Iterativa:", suma_iterativa, "\n")
```

```
## Resultado Suma Iterativa: 5e+13
```

```
system.time({
suma_analitica <- (n * (n + 1)) / 2
})
```

```
##    user  system elapsed
##       0       0       0
```

```
cat("Resultado Suma Analítica:", suma_analitica, "\n")
```

```
## Resultado Suma Analítica: 5e+13
```

Investigar con los gráficos de violin (de la biblioteca micro como se comporta una computadora usando métodos de ordenamiento "burbuja" y compararlo con el método sort nativo de R .

Medir la performance del método kmean de agrupamiento. Realizar una inroducción teórica antes de medir con gráfico de violin.

```
library(microbenchmark)
library(ggplot2)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
# Implementación bubble sort
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n-1)) {
    for (j in 1:(n-i)) {
      if (x[j] > x[j+1]) {
        temp <- x[j]
        x[j] <- x[j+1]
        x[j+1] <- temp
      }
    }
  }
  return(x)
}

# Datos de prueba
set.seed(123)
vec <- sample(1:5000, 500)

# Benchmark
bench <- microbenchmark(
  bubble = bubble_sort(vec),
  sortR = sort(vec),
  times = 30
)

# Convertir a ms
```

```r
df <- as.data.frame(bench) %>%
  mutate(time_ms = time / 1e6)

# Gráfico de violín
ggplot(df, aes(x = expr, y = time_ms)) +
  geom_violin(trim = FALSE, fill = "lightblue", alpha = 0.6) +
  geom_boxplot(width = 0.1, outlier.shape = NA) +
  labs(title = "Bubble Sort vs sort()",
       x = "Método",
       y = "Tiempo (ms)") +
  theme_minimal()
```